



Strategies

Concepts

Strategies

A strategy inherits the `Strategy` class and implements the methods its logic requires.

Capabilities:

- All `Actor` capabilities.
- Order management.

Relationship with actors: The `Strategy` class inherits from `Actor`, which means strategies have access to all actor functionality plus order management capabilities.



We recommend reviewing the [Actors](#) guide before diving into strategy development.

Strategies can be added to Nautilus systems in any [environment contexts](#) and will start sending commands and receiving events based on their logic as soon as the system starts.

With these building blocks of data ingest, event handling, and order management (discussed below), you can build any type of strategy including directional, momentum, re-balancing, pairs, market making, etc.

See the `Strategy` [API Reference](#) for all available methods.

There are two main parts of a Nautilus trading strategy:

- The strategy implementation itself, defined by inheriting the `Strategy` class.
- The *optional* strategy configuration, defined by inheriting the `StrategyConfig` class.



Once a strategy is defined, the same source code can be used for backtesting and live trading.

The main capabilities of a strategy include:

- Historical data requests.
- Live data feed subscriptions.
- Setting time alerts or timers.
- Cache access.
- Portfolio access.
- Creating and managing orders and positions.

Strategy implementation

A trading strategy inherits from `Strategy`, so you must define a constructor. At minimum, initialize the base class:

```
from nautilus_trader.trading.strategy import Strategy

class MyStrategy(Strategy):
    def __init__(self) -> None:
        super().__init__() # <-- the superclass must be called to initialize the strategy
```

From here, you can implement handlers as necessary to perform actions based on state transitions and events.

 Do not call components such as `clock` and `logger` in the `__init__` constructor (which is prior to registration). This is because the systems clock and logging subsystem have not yet been initialized.

Handlers

Handlers are methods on the `Strategy` class that perform actions based on events or state changes. These methods use the `on_*` prefix. Implement any or all of them as your strategy requires.

Multiple handlers exist for similar event types to give you control over granularity. Respond to a specific event with a dedicated handler, or use a generic handler for a range of related events

(using typical switch statement logic). The system calls handlers in sequence from most specific to most general.

Stateful actions

Lifecycle state changes trigger these handlers. Recommendations:

- Use the `on_start` method to initialize your strategy (e.g., fetch instruments, subscribe to data).
- Use the `on_stop` method for cleanup tasks (e.g., cancel open orders, close open positions, unsubscribe from data).

```
def on_start(self) -> None:
def on_stop(self) -> None:
def on_resume(self) -> None:
def on_reset(self) -> None:
def on_dispose(self) -> None:
def on_degrade(self) -> None:
def on_fault(self) -> None:
def on_save(self) -> dict[str, bytes]: # Returns user-defined dictionary of state to be saved
def on_load(self, state: dict[str, bytes]) -> None:
```

Data handling

These handlers receive data updates, including built-in market data and custom user-defined data.

```
from nautilus_trader.core import Data
from nautilus_trader.model import OrderBook
from nautilus_trader.model import Bar
from nautilus_trader.model import QuoteTick
from nautilus_trader.model import TradeTick
from nautilus_trader.model import OrderBookDeltas
from nautilus_trader.model import InstrumentClose
from nautilus_trader.model import InstrumentStatus
from nautilus_trader.model import OptionChainSlice
from nautilus_trader.model import OptionGreeks
from nautilus_trader.model.instruments import Instrument

def on_order_book_deltas(self, deltas: OrderBookDeltas) -> None:
def on_order_book(self, order_book: OrderBook) -> None:
def on_quote_tick(self, tick: QuoteTick) -> None:
def on_trade_tick(self, tick: TradeTick) -> None:
def on_bar(self, bar: Bar) -> None:
def on_instrument(self, instrument: Instrument) -> None:
def on_instrument_status(self, data: InstrumentStatus) -> None:
```

```
def on_instrument_close(self, data: InstrumentClose) -> None:
def on_option_greeks(self, greeks: OptionGreeks) -> None:
def on_option_chain(self, chain: OptionChainSlice) -> None:
def on_historical_data(self, data: Data) -> None:
def on_data(self, data: Data) -> None: # Custom data passed to this handler
def on_signal(self, signal: Data) -> None: # Custom signals passed to this handler
```

Order management

These handlers receive events related to orders. `OrderEvent` type messages are passed to handlers in the following sequence:

1. Specific handler (e.g., `on_order_accepted`, `on_order_rejected`, etc.)
2. `on_order_event(...)`
3. `on_event(...)`

```
from nautilus_trader.model.events import OrderAccepted
from nautilus_trader.model.events import OrderCanceled
from nautilus_trader.model.events import OrderCancelRejected
from nautilus_trader.model.events import OrderDenied
from nautilus_trader.model.events import OrderEmulated
from nautilus_trader.model.events import OrderEvent
from nautilus_trader.model.events import OrderExpired
from nautilus_trader.model.events import OrderFilled
from nautilus_trader.model.events import OrderInitialized
from nautilus_trader.model.events import OrderModifyRejected
from nautilus_trader.model.events import OrderPendingCancel
from nautilus_trader.model.events import OrderPendingUpdate
from nautilus_trader.model.events import OrderRejected
from nautilus_trader.model.events import OrderReleased
from nautilus_trader.model.events import OrderSubmitted
from nautilus_trader.model.events import OrderTriggered
from nautilus_trader.model.events import OrderUpdated

def on_order_initialized(self, event: OrderInitialized) -> None:
def on_order_denied(self, event: OrderDenied) -> None:
def on_order_emulated(self, event: OrderEmulated) -> None:
def on_order_released(self, event: OrderReleased) -> None:
def on_order_submitted(self, event: OrderSubmitted) -> None:
def on_order_rejected(self, event: OrderRejected) -> None:
def on_order_accepted(self, event: OrderAccepted) -> None:
def on_order_canceled(self, event: OrderCanceled) -> None:
def on_order_expired(self, event: OrderExpired) -> None:
def on_order_triggered(self, event: OrderTriggered) -> None:
def on_order_pending_update(self, event: OrderPendingUpdate) -> None:
def on_order_pending_cancel(self, event: OrderPendingCancel) -> None:
def on_order_modify_rejected(self, event: OrderModifyRejected) -> None:
```


Position management

These handlers receive events related to positions. `PositionEvent` type messages are passed to handlers in the following sequence:

1. Specific handler (e.g., `on_position_opened`, `on_position_changed`, etc.)
2. `on_position_event(...)`
3. `on_event(...)`

```
from nautilus_trader.model.events import PositionChanged
from nautilus_trader.model.events import PositionClosed
from nautilus_trader.model.events import PositionEvent
from nautilus_trader.model.events import PositionOpened

def on_position_opened(self, event: PositionOpened) -> None:
def on_position_changed(self, event: PositionChanged) -> None:
def on_position_closed(self, event: PositionClosed) -> None:
def on_position_event(self, event: PositionEvent) -> None: # All position event messages are eventu
```

Generic event handling

This handler will eventually receive all event messages which arrive at the strategy, including those for which no other specific handler exists.

```
from nautilus_trader.core.message import Event

def on_event(self, event: Event) -> None:
```

Handler example

The following example shows a typical `on_start` handler method implementation (taken from the example EMA cross strategy). Here we can see the following:

- Indicators being registered to receive bar updates.
- Historical data being requested (to hydrate the indicators).
- Live data being subscribed to.

```
def on_start(self) -> None:
```

```
"""
Actions to be performed on strategy start.
"""

self.instrument = self.cache.instrument(self.instrument_id)
if self.instrument is None:
    self.log.error(f"Could not find instrument for {self.instrument_id}")
    self.stop() # Transitions strategy to STOPPED state
    return

# Register the indicators for updating
self.register_indicator_forBars(self.bar_type, self.fast_ema)
self.register_indicator_forBars(self.bar_type, self.slow_ema)

# Get historical data
self.request_bars(self.bar_type)

# Subscribe to live data
self.subscribe_bars(self.bar_type)
self.subscribe_quote_ticks(self.instrument_id)
```

Clock and timers

Strategies have access to a `Clock` which provides a number of methods for creating different timestamps, as well as setting time alerts or timers to trigger `TimeEvent`s.

See the `Clock` [API Reference](#) for all available methods.

Current timestamps

While there are multiple ways to obtain current timestamps, here are two commonly used methods as examples:

To get the current UTC timestamp as a tz-aware `pd.Timestamp`:

```
import pandas as pd

now: pd.Timestamp = self.clock.utcnow()
```



To get the current UTC timestamp as nanoseconds since the UNIX epoch:

```
unix_nanos: int = self.clock.timestamp_ns()
```



Time alerts

Time alerts can be set which will result in a `TimeEvent` being dispatched to the `on_event` handler at the specified alert time. In a live context, this might be slightly delayed by a few microseconds.

This example sets a time alert to trigger one minute from the current time:

```
import pandas as pd

# Fire a TimeEvent one minute from now
self.clock.set_time_alert(
    name="MyTimeAlert1",
    alert_time=self.clock.utcnow() + pd.Timedelta(minutes=1),
)
```

Timers

Continuous timers can be set up which will generate a `TimeEvent` at regular intervals until the timer expires or is canceled.

This example sets a timer to fire once per minute, starting immediately:

```
import pandas as pd

# Fire a TimeEvent every minute
self.clock.set_timer(
    name="MyTimer1",
    interval=pd.Timedelta(minutes=1),
)
```

Cache access

The trader's central `Cache` stores data and execution objects (orders, positions, etc). Many methods are available with filtering. Here are some basic use cases.

Fetching data

The following example fetches data from the cache (assuming some instrument ID attribute is assigned). These methods return `None` if the requested data is not available.

```
last_quote = self.cache.quote_tick(self.instrument_id)
last_trade = self.cache.trade_tick(self.instrument_id)
last_bar = self.cache.bar(bar_type)
```



Fetching execution objects

The following example shows how individual order and position objects can be fetched from the cache:

```
order = self.cache.order(client_order_id)
position = self.cache.position(position_id)
```



See the [Cache API Reference](#) for all available methods.

Portfolio access

The trader's central [Portfolio](#) provides account and positional information. The following shows a general outline of available methods.

Account and positional information

```
import decimal

from nautilus_trader.accounting.accounts.base import Account
from nautilus_trader.model import Venue
from nautilus_trader.model import Currency
from nautilus_trader.model import Money
from nautilus_trader.model import InstrumentId

def account(self, venue: Venue) -> Account

def balances_locked(self, venue: Venue) -> dict[Currency, Money]
def margins_init(self, venue: Venue) -> dict[Currency, Money]
def margins_maint(self, venue: Venue) -> dict[Currency, Money]
def unrealized_pnls(self, venue: Venue) -> dict[Currency, Money]
def realized_pnls(self, venue: Venue) -> dict[Currency, Money]
def net_exposures(self, venue: Venue) -> dict[Currency, Money]

def unrealized_pnl(self, instrument_id: InstrumentId) -> Money
def realized_pnl(self, instrument_id: InstrumentId) -> Money
def net_exposure(self, instrument_id: InstrumentId) -> Money
def net_position(self, instrument_id: InstrumentId) -> decimal.Decimal

def is_net_long(self, instrument_id: InstrumentId) -> bool
```



```
def is_net_short(self, instrument_id: InstrumentId) -> bool
def is_flat(self, instrument_id: InstrumentId) -> bool
def is_completely_flat(self) -> bool
```

See the [Portfolio API Reference](#) for all available methods.

Reports and analysis

The [Portfolio](#) also exposes a [PortfolioAnalyzer](#), which accepts a flexible amount of data (to accommodate different lookback windows). The analyzer tracks and generates performance metrics and statistics.

See the [PortfolioAnalyzer API Reference](#) and [Portfolio statistics](#) guide.

Trading commands

The following trading commands are available for order management. See also the [Execution](#) guide for the full flow through the system.

Submitting orders

An [OrderFactory](#) is provided on the base class for every [Strategy](#) as a convenience, reducing the amount of boilerplate required to create different [Order](#) objects (although these objects can still be initialized directly with the [Order.__init__\(...\)](#) constructor if the trader prefers).

The component a [SubmitOrder](#) or [SubmitOrderList](#) command will flow to for execution depends on the following:

- If an [emulation_trigger](#) is specified, the command will *firstly* be sent to the [OrderEmulator](#).
- If an [exec_algorithm_id](#) is specified (with no [emulation_trigger](#)), the command will *firstly* be sent to the relevant [ExecAlgorithm](#).
- Otherwise, the command will *firstly* be sent to the [RiskEngine](#).

This example submits a [LIMIT](#) BUY order for emulation (see [Emulated Orders](#)):

```
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.model.enums import TriggerType
from nautilus_trader.model.orders import LimitOrder
```



```
def buy(self) -> None:
    """
    Users simple buy method (example).
    """
    order: LimitOrder = self.order_factory.limit(
        instrument_id=self.instrument_id,
        order_side=OrderSide.BUY,
        quantity=self.instrument.make_qty(self.trade_size),
        price=self.instrument.make_price(5000.00),
        emulation_trigger=TriggerType.LAST_PRICE,
    )

    self.submit_order(order)
```

 You can specify both order emulation and an execution algorithm. In this case, the order is first sent to the `OrderEmulator`, and upon release is then routed to the `ExecAlgorithm`.

This example submits a `MARKET` BUY order to a TWAP execution algorithm:

```
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.model.enums import TimeInForce
from nautilus_trader.model import ExecAlgorithmId

def buy(self) -> None:
    """
    Users simple buy method (example).
    """
    order: MarketOrder = self.order_factory.market(
        instrument_id=self.instrument_id,
        order_side=OrderSide.BUY,
        quantity=self.instrument.make_qty(self.trade_size),
        time_in_force=TimeInForce.FOK,
        exec_algorithm_id=ExecAlgorithmId("TWAP"),
        exec_algorithm_params={"horizon_secs": 20, "interval_secs": 2.5},
    )

    self.submit_order(order)
```

Canceling orders

Orders can be canceled individually, as a batch, or all orders for an instrument (with an optional side filter).

If the order is already *closed* or already pending cancel, then a warning will be logged.

If the order is currently *open* then the status will become `PENDING_CANCEL`.

The component a `CancelOrder`, `CancelAllOrders` or `BatchCancelOrders` command will flow to for execution depends on the following:

- If the order is currently emulated, the command will *firstly* be sent to the `OrderEmulator`.
- If an `exec_algorithm_id` is specified (with no `emulation_trigger`), and the order is still active within the local system, the command will *firstly* be sent to the relevant `ExecAlgorithm`.
- Otherwise, the order will *firstly* be sent to the `ExecutionEngine`.

 Any managed GTD timer will also be canceled after the command has left the strategy.

The following shows how to cancel an individual order:

```
self.cancel_order(order)
```



The following shows how to cancel a batch of orders:

```
from nautilus_trader.model.orders import Order

my_order_list: list[Order] = [order1, order2, order3]
self.cancel_orders(my_order_list)
```



The following shows how to cancel all orders:

```
self.cancel_all_orders()
```



Modifying orders

Orders can be modified individually when emulated, or *open* on a venue (if supported).

If the order is already *closed* or already pending cancel, then a warning will be logged. If the order is currently *open* then the status will become `PENDING_UPDATE`.

 At least one value must differ from the original order for the command to be valid.

The component a `ModifyOrder` command will flow to for execution depends on the following:

- If the order is currently emulated, the command will *firstly* be sent to the `OrderEmulator`.
- Otherwise, the order will *firstly* be sent to the `RiskEngine`.

 Once an order is under the control of an execution algorithm, it cannot be directly modified by a strategy (only canceled).

The following shows how to modify the size of `LIMIT` BUY order currently *open* on a venue:

```
from nautilus_trader.model import Quantity

new_quantity: Quantity = Quantity.from_int(5)
self.modify_order(order, new_quantity)
```

 The price and trigger price can also be modified (when emulated or supported by a venue).

Market exit

The `market_exit()` method provides a graceful way to exit all positions and cancel all orders for a strategy. The strategy remains running after the exit completes, allowing you to re-enter positions later if desired.

```
self.market_exit()
```

The market exit process:

1. Cancels all open and in-flight orders for the strategy.
2. Closes all open positions with market orders.
3. Periodically checks (at `market_exit_interval_ms`) until all orders resolve and positions close.
4. Calls `post_market_exit()` once flat, or after `market_exit_max_attempts` is reached.

Two hooks are available for custom logic:

- `on_market_exit()` - Called when the exit process begins.
- `post_market_exit()` - Called when the exit process completes.

```
class MyStrategy(Strategy):
    def on_market_exit(self) -> None:
        self.log.info("Beginning market exit...")

    def post_market_exit(self) -> None:
        self.log.info("Market exit complete")
```

During a market exit, non-reduce-only orders are automatically denied. For order lists, if any order in the list is non-reduce-only, the entire list is denied to preserve list semantics (e.g., bracket orders with interdependencies).

To check if an exit is in progress (e.g., to skip order submission logic), use `is_exiting()`:

```
def on_quote_tick(self, tick: QuoteTick) -> None:
    if self.is_exiting():
        return # Skip order logic during exit
    # ... normal order logic
```

To automatically perform a market exit when the strategy is stopped, set `manage_stop=True`:

```
config = StrategyConfig(manage_stop=True)
```

With this option, calling `stop()` will first perform a market exit, then stop the strategy once flat.

Configuration options in `StrategyConfig`:

- `manage_stop` (default: False) - If True, `stop()` performs a market exit before stopping.
- `market_exit_interval_ms` (default: 100) - Interval between exit completion checks.
- `market_exit_max_attempts` (default: 100) - Maximum checks before completing the exit.
- `market_exit_time_in_force` (default: None/GTC) - Time in force for closing market orders.
- `market_exit_reduce_only` (default: True) - If closing market orders should be reduce only.

Strategy configuration

A separate configuration class gives full flexibility over where and how a strategy is instantiated. Configurations serialize over the wire, enabling distributed backtesting and remote live trading.

This is opt-in. You can skip configuration and pass parameters directly to your strategy constructor. If you want distributed backtests or remote live trading, define a configuration.

Here is an example configuration:

```
from decimal import Decimal
from nautilus_trader.config import StrategyConfig
from nautilus_trader.model import Bar, BarType
from nautilus_trader.model import InstrumentId
from nautilus_trader.trading.strategy import Strategy

# Configuration definition
class MyStrategyConfig(StrategyConfig):
    instrument_id: InstrumentId # example value: "ETHUSD-T-PERP.BINANCE"
    bar_type: BarType          # example value: "ETHUSD-T-PERP.BINANCE-15-MINUTE[LAST]-EXTERNAL"
    fast_ema_period: int = 10
    slow_ema_period: int = 20
    trade_size: Decimal
    order_id_tag: str

# Strategy definition
class MyStrategy(Strategy):
    def __init__(self, config: MyStrategyConfig) -> None:
        # Always initialize the parent Strategy class
        # After this, configuration is stored and available via `self.config`
        super().__init__(config)

        # Custom state variables
        self.time_started = None
        self.count_of_processedBars: int = 0

    def on_start(self) -> None:
        self.time_started = self.clock.utc_now() # Remember time, when strategy started
        self.subscribeBars(self.config.bar_type) # See how configuration data are exposed via `self.config`
```

Access configuration values through `self.config`. This provides clear separation between:

- Configuration data (accessed via `self.config`):

- Contains initial settings, that define how the strategy works.
- Example: `self.config.trade_size`, `self.config.instrument_id`
- Strategy state variables (as direct attributes):
 - Track any custom state of the strategy.
 - Example: `self.time_started`, `self.count_of_processed_bars`

This separation makes code easier to understand and maintain.



Even though it often makes sense to define a strategy which will trade a single instrument. The number of instruments a single strategy can work with is only limited by machine resources.

Managed GTD expiry

It's possible for the strategy to manage expiry for orders with a time in force of GTD (*Good 'till Date*). This may be desirable if the exchange/broker does not support this time in force option, or for any reason you prefer the strategy to manage this.

To use this option, pass `manage_gtd_expiry=True` to your `StrategyConfig`. When an order is submitted with a time in force of GTD, the strategy will automatically start an internal time alert. Once the internal GTD time alert is reached, the order will be canceled (if not already *closed*).

Some venues (such as Binance Futures) support the GTD time in force, so to avoid conflicts when using `managed_gtd_expiry` you should set `use_gtd=False` for your execution client config.

Multiple strategies

If you intend running multiple instances of the same strategy, with different configurations (such as trading different instruments), then each instance needs a unique strategy ID and order ID tag.

If `strategy_id` is not supplied, the platform builds the strategy ID from the strategy class name and an order ID tag. The tag can be supplied with `order_id_tag`; otherwise registration assigns the next numeric tag, starting with `000`. For example, the above config results in a strategy ID of `MyStrategy-001`.

If `strategy_id` is supplied with `order_id_tag`, Rust appends the tag to the runtime strategy ID unless the ID already ends with that tag. For example, `strategy_id=MyStrategy-PRIMARY` with `order_id_tag=ABC` becomes `MyStrategy-PRIMARY-ABC`. If `order_id_tag` is omitted, Rust uses the final hyphen-separated part of `strategy_id` as the order ID tag.



The platform has built-in safety measures: if two strategies share a duplicated strategy ID, a `RuntimeError` is raised during registration indicating the strategy ID is already registered.

The reason for this is that the system must be able to identify which strategy various commands and events belong to. The order ID tag also keeps generated client order IDs unique across strategies for the same trader.

info Rust implementation Rust treats `StrategyConfig` as immutable construction input. The runtime `StrategyId` carries the order ID tag, matching Python/Cython behavior. This keeps actor registration, client order ID generation, order list ID generation, and position ID generation aligned through `strategy_id.get_tag()`.

If `strategy_id` is omitted, `order_id_tag` overrides the generated suffix, for example `MyStrategy-ABC`.

...

See the `StrategyId` [API Reference](#) for further details.

Related guides

- [Actors](#) - Base class that strategies extend.
- [Events](#) - Event types and handler dispatch.
- [Orders](#) - Order types and management from strategies.
- [Backtesting](#) - Test strategies with historical data.

< Actors

Previous Page

Instruments >

Next Page